



Design and Analysis of Algorithms

Vandana M L

Department of Computer Science & Engineering

DESIGN AND ANALYSIS OF ALGORITHMS

Mathematical Analysis of Recursive Algorithms

Slides courtesy of **Anany Levitin**

Vandana M L

Department of Computer Science & Engineering

Design and Analysis of Algorithms

Steps in Mathematical Analysis of Recursive Algorithms



- Decide on parameter n indicating input size
- Identify algorithm's basic operation
- If the number of times the basic operation is executed varies with different inputs of same sizes, investigate worst, average, and best case efficiency separately
- Set up a recurrence relation and initial condition(s) for $C(n)$ -the number of times the basic operation will be executed for an input of size n
- Solve the recurrence or estimate the order of magnitude of the solution

Design and Analysis of Algorithms

Important Recurrence Types



➤ Decrease-by-one recurrences

A decrease-by-one algorithm solves a problem by exploiting a relationship between a given instance of size n and a smaller size $n - 1$.

Example: $n!$

The recurrence equation has the form

$$T(n) = T(n-1) + f(n)$$

➤ Decrease-by-a-constant-factor recurrences

A decrease-by-a-constant algorithm solves a problem by dividing its given instance of size n into several smaller instances of size n/b , solving each of them recursively, and then, if necessary, combining the solutions to the smaller instances into a solution to the given instance.

Example: binary search.

The recurrence has the form

$$T(n) = aT(n/b) + f(n)$$

Design and Analysis of Algorithms

Decrease-by-one Recurrences

- One (constant) operation reduces problem size by one.

$$T(n) = T(n-1) + c \qquad T(1) = d$$

Solution: $T(n) = (n-1)c + d$ linear

- A pass through input reduces problem size by one.

$$T(n) = T(n-1) + c n \qquad T(1) = d$$

Solution: $T(n) = [n(n+1)/2 - 1] c + d$ quadratic

Design and Analysis of Algorithms

Methods to solve recurrences

- Substitution Method
 - Mathematical Induction
 - Backward substitution
- Recursion Tree Method
- Master Method (Decrease by constant factor recurrences)

Design and Analysis of Algorithms

Recursive Evaluation of $n!$

$$n! = 1 * 2 * \dots * (n-1) * n \quad \text{for } n \geq 1 \quad \text{and} \quad 0! = 1$$

Recursive definition of $n!$:

$$F(n) = F(n-1) * n \quad \text{for } n \geq 1 \quad \text{input size?}$$

$$F(0) = 1$$

basic operation?

ALGORITHM $F(n)$

//Computes $n!$ recursively

//Input: A nonnegative integer n

//Output: The value of $n!$

if $n = 0$ **return** 1

else return $F(n - 1) * n$

Best/Worst/Average Case?

$$M(n) = M(n - 1) + 1 \quad \text{for } n > 0,$$

$$M(0) = 0.$$

$$M(n-1) = M(n-2) + 1; \quad M(n-2) = M(n-3) + 1$$

$$M(n) = n$$

Overall time Complexity: $\Theta(n)$

Design and Analysis of Algorithms

Counting number of binary digits in binary representation of a number

ALGORITHM *BinRec(n)*

//Input: A positive decimal integer n

//Output: The number of binary digits in n 's binary representation

if $n = 1$ return 1

else return $\text{BinRec}(\lfloor n/2 \rfloor) + 1$

input size?

basic operation?

Best/Worst/Average Case?

$$A(2^k) = A(2^{k-1}) + 1 \quad \text{for } k > 0,$$

$$A(2^0) = 0.$$

$$A(2^k) = A(2^{k-1}) + 1 \quad \text{substitute } A(2^{k-1}) = A(2^{k-2}) + 1$$

$$= [A(2^{k-2}) + 1] + 1 = A(2^{k-2}) + 2 \quad \text{substitute } A(2^{k-2}) = A(2^{k-3}) + 1$$

$$= [A(2^{k-3}) + 1] + 2 = A(2^{k-3}) + 3 \quad \dots$$

...

$$= A(2^{k-i}) + i$$

...

$$= A(2^{k-k}) + k.$$

$$A(n) = \log_2 n \in \Theta(\log n).$$

Design and Analysis of Algorithms

Tower of Hanoi

Algorithm TowerOfHanoi(n, Src, Aux, Dst)

if (n = 0)

return

TowerOfHanoi(n-1, Src, Dst, Aux)

Move disk n from Src to Dst

TowerOfHanoi(n-1, Aux, Src, Dst)

Input Size: n

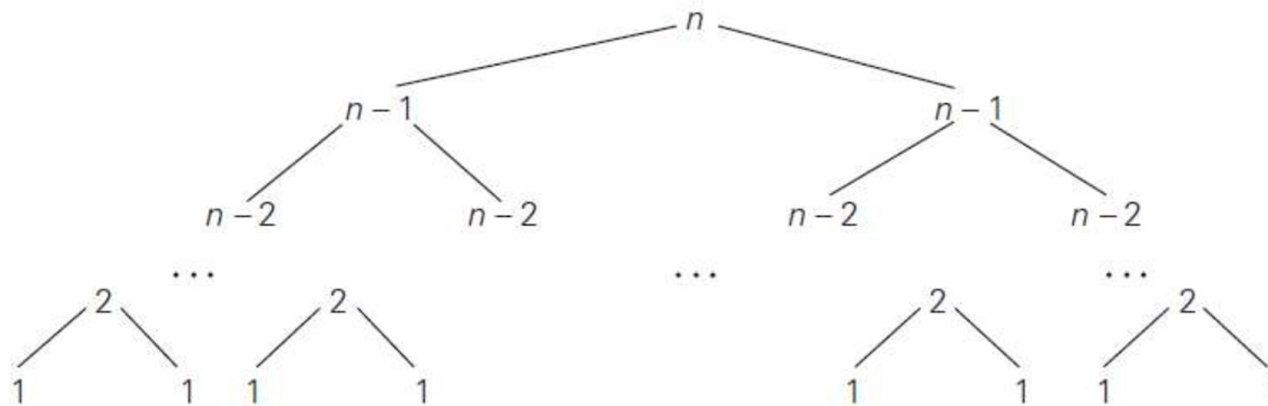
Basic Operation : **Move disk n from Src to Dst**

C(n) = 2C(n-1) + 1 for $n > 0$ and $C(0)=0$

= $2^n - 1 \in \Theta(2^n)$

Design and Analysis of Algorithms

Tower of Hanoi : Tree of Recursive calls



$$C(n) = \sum_{l=0}^{n-1} 2^l = 2^n - 1$$

In the recursive factorial algorithm $F(n)$, why is the number of multiplications chosen as the basic operation?

- A) Multiplication is asymptotically dominant over recursion
- B) It directly reflects the size of the input
- C) It is executed exactly once per recursive call (except the base case)
- D) It is more expensive than function calls

Answer C) It is executed exactly once per recursive call (except the base case)

Which recurrence correctly models the number of multiplications $M(n)$ required to compute $n!$ using the recursive algorithm?

- A) $M(n) = M(n-1) + n, M(0)=0$
- B) $M(n) = M(n-1)+1, M(0)=0$
- C) $M(n) = nM(n-1), M(0)=1$
- d) $M(n)=M(n/2)+1, M(1)=0$

Answer B) $M(n) = M(n-1)+1, M(0)=0$

In the recurrence relation for the factorial function, why is the base case $M(0)=0$ fundamentally necessary?

- A) It guarantees the numerical result of the factorial is accurate
- B) It defines the stopping point for backward substitution in the recursion
- C) It reduces the total number of recursive calls made
- D) It prevents excessive memory usage and stack overflow

Answer B) It determines where backward substitution must stop

Applying backward substitution to $M(n) = M(n - 1) + 1$, $M(0) = 0$, yields which exact solution?

- A) $M(n) = n!$
- B) $M(n) = 2^n$
- C) $M(n) = n$
- D) $M(n) = \log n$

Answer C) $M(n) = n$

Which statement correctly distinguishes the two recursive functions discussed in the factorial example?

- A) Both define the same recurrence
- B) $F(n)$ defines a mathematical function, while $M(n)$ defines algorithmic cost
- C) $M(n)$ grows faster than $F(n)$
- D) $F(n)$ is solved using recursion trees

Answer: B) $F(n)$ defines a mathematical function, while $M(n)$ defines algorithmic cost

What key insight does the recursion tree for the Tower of Hanoi algorithm provide?

- A) It reduces the recurrence to a summation
- B) It visually confirms the exponential number of recursive calls
- C) It improves the asymptotic efficiency
- D) It eliminates the need for backward substitution

B) It visually confirms the exponential number of recursive calls

Why does the factorial algorithm's recurrence for cost differ fundamentally from the recurrence defining the factorial function itself?

- A) One is linear while the other is exponential
- B) One models mathematical growth, the other models algorithmic work
- C) One uses recursion, the other does not
- D) One ignores base cases

B) One models mathematical growth, the other models algorithmic work

Which factor primarily determines the depth of recursion in the recursive factorial algorithm?

- A) The magnitude of intermediate multiplication results
- B) The value of the input parameter n
- C) The number of basic operations
- D) The size of the recursion stack

Answer : B) The value of the input parameter n

ALGORITHM *Riddle*($A[0..n - 1]$)

//Input: An array $A[0..n - 1]$ of real numbers

if $n = 1$ return $A[0]$

else $temp \leftarrow Riddle(A[0..n - 2])$

if $temp \leq A[n - 1]$ return $temp$

else return $A[n - 1]$

The recursive algorithm *Riddle*($A[0..n-1]$) processes an array of real numbers. What does the algorithm return, and what is its worst-case time complexity expressed in terms of n ?

- A) Returns the **maximum** element; Complexity $O(n)$
- B) Returns the **minimum** element; Complexity $O(n)$
- C) Returns the **second smallest** element; Complexity $O(n^2)$
- D) Returns the **last element** if it is smaller than the previous minimum; Complexity $O(n \log n)$

Answer B) Returns the **minimum** element; Complexity $O(n)$

ALGORITHM *Riddle*($A[0..n - 1]$)

//Input: An array $A[0..n - 1]$ of real numbers

if $n = 1$ return $A[0]$

else $temp \leftarrow Riddle(A[0..n - 2])$

if $temp \leq A[n - 1]$ return $temp$

else return $A[n - 1]$

The recursive algorithm *Riddle*($A[0..n-1]$) processes an array of real numbers. What does the algorithm return, and what is its worst-case time complexity expressed in terms of n ?

- A) Returns the **maximum** element; Complexity $O(n)$
- B) Returns the **minimum** element; Complexity $O(n)$
- C) Returns the **second smallest** element; Complexity $O(n^2)$
- D) Returns the **last element** if it is smaller than the previous minimum; Complexity $O(n \log n)$

Answer B) Returns the **minimum** element; Complexity $O(n)$



THANK YOU

Vandana M L

Department of Computer Science & Engineering

vandanamd@pes.edu